This guide breaks down every program from **Experiment 1 to Experiment 10** in this repository. It is tailored for writing these programs, explaining the logic, and answering viva/theory questions in an exam.

Table of Contents

1. Exp 1: 8086 Basics — "Hello World" & Addition
2. Exp 2: 8086 Memory Block Transfer (Within & Across Segments)
3. Exp 3: 8086 String Operations (Length, Compare, Palindrome, Reverse)
4. Exp 4: Booth's Multiplication Algorithm
5. Exp 5: Division Algorithms (Restoring & Non-Restoring)
6. Exp 6: Amdahl's Law (Speedup & Efficiency)
7. Exp 7: Page Replacement Policies (FIFO & LRU)
8. Exp 8: Cache Memory Direct Mapping
9. Exp 10: Pipelining Simulation & Hazard Stalls

Experiment 1: 8086 Basics

Common 8086 Structure to Remember for the Exam

Every standard MASM/TASM 8086 program follows this skeleton:

```
.MODEL SMALL          ; Allocates memory model (code <= 64KB, data <= 64KB)
.STACK 100H           ; Reserves 256 bytes (100h) for the stack
.DATA                 ; Start of data segment (variables declared here)
    ; variables
.CODE                 ; Start of code segment (instructions written here)
MAIN PROC
    MOV AX, @DATA     ; 8086 cannot load @DATA directly into DS
    MOV DS, AX        ; Initialize Data Segment register
    ; your logic
    MOV AH, 4CH       ; DOS interrupt function 4CH = exit/terminate process
    INT 21H
MAIN ENDP
END MAIN
```

Program 1.1: Display "Hello World" ([HELLO.ASM](file:///home/mint/Documents/COA/Week%201/HELLO.ASM))

```
.MODEL SMALL
.STACK 100H

.DATA
```

```
    MSG DB 'Hello World$'       ; '$' is the string terminator required by
DOS function 09H

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX                  ; Initialize DS

    LEA DX, MSG                 ; Load Effective Address of MSG into DX
    MOV AH, 09H                 ; DOS function 09H: Display string starting
at DS:DX
    INT 21H                     ; Call DOS interrupt

    MOV AH, 4CH                 ; DOS function 4CH: Exit program
    INT 21H
MAIN ENDP
END MAIN
```

- **How it works**:
  - `MSG DB 'Hello World$'`: Defines bytes (`DB`). In DOS, interrupt `21H` with `AH = 09H` prints characters until it encounters the `$` character.
  - `LEA DX, MSG`: Loads the offset address of `MSG` into `DX`.
  - `INT 21H`: Triggers DOS service routines.

Program 1.2: Add Two Numbers ([ADD2NUMS.ASM](file:///home/mint/Documents/COA/Week%201/ADD2NUMS.ASM))

16-Bit Addition (As per syllabus):

```
.MODEL SMALL
.STACK 100H

.DATA
    NUM1 DW 1234H     ; DW = Define Word (16 bits / 2 bytes)
    NUM2 DW 5678H
    SUM  DW ?         ; '?' reserves uninitialized memory

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    MOV AX, NUM1      ; Load 16-bit NUM1 into accumulator AX
    ADD AX, NUM2      ; AX = AX + NUM2
    MOV SUM, AX       ; Store result in memory location SUM

    MOV AH, 4CH       ; Exit to DOS
    INT 21H
```

```
MAIN ENDP
END MAIN
```

(Note: If asked for 8-bit addition, replace `DW` with `DB` and `AX` with `AL`.)

Experiment 2: Memory Block Transfer

Key Instructions to Know:

- `MOVSW` (Move String Word): Copies 1 word (16 bits) from source address `DS:SI` to destination address `ES:DI`. After copying, it automatically increments `SI` and `DI` by 2 (if Direction Flag `DF = 0`).
- `CLD` (Clear Direction Flag): Ensures `DF = 0` (forward scanning / auto-increment).
- `REP`: Repeats the string instruction `CX` times, decrementing `CX` after each transfer until `CX = 0`.

Program 2.1: Transfer 10 Words within the Same Segment (`1000:0000` to `1000:2000`) ([`COPYINSEG.ASM`](file:///home/mint/Documents/COA/Week%202/COPYINSEG.ASM))

```
.MODEL SMALL
.STACK 100H
.CODE
MAIN PROC
    MOV AX, 1000H
    MOV DS, AX          ; Source Segment DS = 1000H
    MOV ES, AX          ; Destination Segment ES = 1000H

    MOV SI, 0000H       ; Source Offset SI = 0000H
    MOV DI, 2000H       ; Destination Offset DI = 2000H
    MOV CX, 10          ; Counter = 10 words (each word is 16 bits)

    CLD                 ; Clear Direction Flag (auto-increment SI & DI)
    REP MOVSW           ; Repeat copy word CX times

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Program 2.2: Transfer 10 Words Across Segments (`1000:0000` to `2000:0000`) ([`COPYXSEG.ASM`](file:///home/mint/Documents/COA/Week%202/COPYXSEG.ASM))

```
.MODEL SMALL
.STACK 100H
.CODE
MAIN PROC
```

```
    MOV AX, 1000H
    MOV DS, AX          ; Source Segment DS = 1000H

    MOV AX, 2000H
    MOV ES, AX          ; Destination Segment ES = 2000H

    XOR SI, SI          ; SI = 0000H (XORing register with itself clears it
to 0)
    XOR DI, DI          ; DI = 0000H
    MOV CX, 10          ; Count = 10 words

    CLD
    REP MOVSW           ; Copies from 1000:0000 to 2000:0000

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Experiment 3: String Operations

Program 3.1: Length of a String ([STRLEN.ASM](file:///home/mint/Documents/COA/Week%203/STRLEN.ASM))

- **Concept**: Use `LODSB` (loads byte at `DS:SI` into `AL` and increments `SI`). Loop until character matches `'$'`.

```
.MODEL SMALL
.STACK 100H
.DATA
    STR1 DB "Hello World$"
    LEN  DB 0

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA SI, STR1        ; SI points to start of STR1
    XOR CX, CX          ; CX = 0 (Length counter)

L1:
    LODSB               ; AL = [SI], SI = SI + 1
    CMP AL, '$'         ; Check for end of string
    JE DONE             ; If equal to '$', exit loop
    INC CX              ; Otherwise, increment counter
    JMP L1

DONE:
```

```
    MOV LEN, CL        ; Store length in variable LEN
    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Program 3.2: Check if Two Strings are Equal ([CMPSTR.ASM](file:///home/mint/Documents/COA/Week%203/CMPSTR.ASM))

- **Concept**: Use `CMPSB` (compares byte at `DS:SI` with byte at `ES:DI`) combined with prefix `REPE` (Repeat while Equal).

```
.MODEL SMALL
.STACK 100H
.DATA
    STRING1  DB 'HELLO$'
    LEN1     EQU $ - STRING1 - 1    ; Assembler computes length excluding
'$'
    STRING2  DB 'HELLO$'
    EQUALMSG DB 'Strings are equal.$'
    NEQMSG   DB 'Strings are NOT equal.$'

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX
    MOV ES, AX         ; ES must point to data segment for CMPSB

    LEA SI, STRING1    ; Source string
    LEA DI, STRING2    ; Destination string
    MOV CX, LEN1       ; Number of characters to compare
    CLD                ; Left-to-right scan

    REPE CMPSB        ; Repeat while bytes are equal and CX != 0
    JNE NEQUAL         ; If Zero Flag = 0 (mismatch occurred), jump to
NEQUAL

    LEA DX, EQUALMSG
    JMP SHOW

NEQUAL:
    LEA DX, NEQMSG

SHOW:
    MOV AH, 09H        ; Display string in DX
    INT 21H

    MOV AH, 4CH
    INT 21H
```

```
MAIN ENDP
END MAIN
```

Program 3.3: Check if a String is a Palindrome ([PALIN.ASM](file:///home/mint/Documents/COA/Week%203/PALIN.ASM))

- **Concept**: Two pointers — `SI` starts at character 0, `DI` starts at the last character (`LEN - 1`). Compare `[SI]` and `[DI]`, then increment `SI` and decrement `DI`. Loop `LEN / 2` times.

```
.MODEL SMALL
.STACK 100H
.DATA
    STR1 DB 'MADAM$'
    LEN  EQU $ - STR1 - 1
    YES  DB 'Palindrome.$'
    NO   DB 'Not a palindrome.$'

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX
    MOV ES, AX

    LEA SI, STR1              ; Front pointer
    LEA DI, STR1 + LEN - 1    ; Back pointer
    MOV CX, LEN / 2           ; Check half the string length
    CLD

CHECK:
    MOV AL, [SI]
    CMP AL, ES:[DI]
    JNE DIFFERENT             ; Mismatch found -> Not palindrome
    INC SI                    ; Move forward
    DEC DI                    ; Move backward
    LOOP CHECK                ; Decrements CX, jumps to CHECK if CX != 0

    LEA DX, YES               ; All characters matched
    JMP SHOW

DIFFERENT:
    LEA DX, NO

SHOW:
    MOV AH, 09H
    INT 21H
    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Program 3.4: Reverse a String ([REVERSE.ASM](file:///home/mint/Documents/COA/Week%203/REVERSE.ASM))

- **Concept**: Read from the end of `STR1` (`SI = STR1 + LEN - 1`) and write sequentially to the start of `REV` (`DI = REV`).

```
.MODEL SMALL
.STACK 100H
.DATA
    STR1 DB 'HELLO$'
    LEN  EQU $ - STR1 - 1
    REV  DB LEN DUP (?)       ; Reserve LEN bytes for reversed output
         DB '$'               ; Terminator for DOS display

.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA SI, STR1 + LEN - 1    ; Point to last character of source
    LEA DI, REV               ; Point to first byte of destination
    MOV CX, LEN               ; Counter = number of characters
    CLD

COPY:
    MOV AL, [SI]              ; Fetch character from end
    MOV [DI], AL              ; Write to destination
    DEC SI                    ; Move backwards in source
    INC DI                    ; Move forwards in destination
    LOOP COPY

    LEA DX, REV               ; Print reversed string
    MOV AH, 09H
    INT 21H

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Experiment 4: Booth's Multiplication Algorithm

Concept & Algorithm

Booth's Algorithm multiplies signed binary integers in two's complement form.

- Registers used:

    - M = Multiplicand
    - Q = Multiplier
    - A = Accumulator (initialized to 0)
    - Q_{-1} = 1-bit register (initialized to 0)
    - Count = Number of bits (n)
- Look at the pair (Q_0, Q_{-1}):
    - `1 0`: $A \leftarrow A - M$
    - `0 1`: $A \leftarrow A + M$
    - `0 0` or `1 1`: No arithmetic operation
- **Arithmetic Right Shift (ASR)** of $[A, Q, Q_{-1}]$:
    - Sign bit of A is preserved ($A_{n-1} \rightarrow A_{n-1}$).
    - $A_0 \rightarrow Q_{n-1}$, $Q_0 \rightarrow Q_{-1}$.

Code ([`boothAlgo.c`](file:///home/mint/Documents/COA/Week%204/boothAlgo.c) / [`BoothAlgorithm.java`](file:///home/mint/Documents/COA/Week%204/BoothAlgorithm.java))

```c
#include <stdio.h>

int main() {
    int M, Q;
    int A = 0, Q_1 = 0;
    int count = 32;

    printf("Enter Multiplicand (M): ");
    scanf("%d", &M);
    printf("Enter Multiplier (Q): ");
    scanf("%d", &Q);

    while (count > 0) {
        int Q_0 = Q & 1; // Extract least significant bit of Q

        // 1. Inspect (Q_0, Q_1)
        if (Q_0 == 1 && Q_1 == 0) {
            A = A - M;
        } else if (Q_0 == 0 && Q_1 == 1) {
            A = A + M;
        }

        // 2. Arithmetic Right Shift [A, Q, Q_1]
        Q_1 = Q & 1;                    // Current LSB of Q becomes Q_1
        Q = (Q >> 1) & ~(1 << 31);      // Logical shift Q right, clear MSB
        Q |= (A & 1) << 31;             // Transfer LSB of A into MSB of Q
        A = A >> 1;                     // Arithmetic shift of A (preserves
sign)

        count--;
    }
```

```
    // For 32-bit registers, Q holds the lower 32 bits of the product
    printf("Decimal Product: %d\n", Q);
    return 0;
}
```

Experiment 5: Division Algorithms

Division calculates: $\text{Dividend } (Q) \div \text{Divisor } (M) \rightarrow \text{Quotient } (Q), \text{Remainder } (A)$.

5.1: Restoring Division Algorithm ([restoring.c](file:///home/mint/Documents/COA/Week%205/restoring.c))

Logic:

1. Initialize $A = 0$, $\text{Count} = n$.
2. Shift Left $[A, Q]$.
3. Subtract divisor: $A = A - M$.
4. Check sign of A:
   - If $A < 0$ (most significant bit is 1): Subtraction failed. Set $Q_0 = 0$ and **restore** $A = A + M$.
   - If $A \geq 0$: Set $Q_0 = 1$.
5. Decrement count until 0.

```
#include <stdio.h>

int main() {
    int N = 32;
    unsigned A = 0, M, Q;

    printf("Enter Dividend (Q): ");
    scanf("%u", &Q);
    printf("Enter Divisor (M): ");
    scanf("%u", &M);

    while (N > 0) {
        // Shift left combined [A, Q]:
        // A receives the 31st bit (MSB) of Q, Q shifts left by 1
        A = (A << 1) | ((Q >> 31) & 1U);
        Q = Q << 1;

        A = A - M; // Trial subtraction

        if ((A >> 31) & 1U) { // If A is negative
            Q &= ~1U;         // Set Q's LSB (Q0) to 0
            A = A + M;        // RESTORE A
        } else {
            Q |= 1U;          // Set Q's LSB (Q0) to 1
        }
        N--;
```

```
    }

    printf("Quotient (Q): %u\n", Q);
    printf("Remainder (A): %u\n", A);
    return 0;
}
```

5.2: Non-Restoring Division Algorithm ([nonrestoring.c](file:///home/mint/Documents/COA/Week%205/nonrestoring.c))

Logic:

Avoids the restorative addition step in the loop:

1. Initialize $A = 0$, $\text{Count} = n$.
2. In each cycle:
   - If $A \geq 0$: Shift Left $[A, Q]$, then $A = A - M$.
   - If $A < 0$: Shift Left $[A, Q]$, then $A = A + M$.
3. Check sign of new A:
   - If $A \geq 0 \implies Q_0 = 1$.
   - If $A < 0 \implies Q_0 = 0$.
4. **Final step (after loop ends)**: If $A < 0$, do a one-time correction: $A = A + M$.

```
#include <stdio.h>

int main() {
    int N = 32;
    unsigned A = 0, M, Q;

    printf("Enter Dividend (Q): ");
    scanf("%u", &Q);
    printf("Enter Divisor (M): ");
    scanf("%u", &M);

    while (N > 0) {
        // Determine whether to Add or Subtract based on sign of A before
shifting
        int a_neg = (A >> 31) & 1U;

        // Shift Left [A, Q]
        A = (A << 1) | ((Q >> 31) & 1U);
        Q = Q << 1;

        if (a_neg) {
            A = A + M;
        } else {
            A = A - M;
        }
```

```
        // Set Q0 based on sign of current A
        if ((A >> 31) & 1U) {
            Q &= ~1U; // Q0 = 0
        } else {
            Q |= 1U;  // Q0 = 1
        }
        N--;
    }

    // Final correction if remainder is negative
    if ((A >> 31) & 1U) {
        A = A + M;
    }

    printf("Quotient (Q): %u\n", Q);
    printf("Remainder (A): %u\n", A);
    return 0;
}
```

Experiment 6: Amdahl's Law

Formulas

Amdahl's Law predicts theoretical speedup when using multiple processors:

$$\text{Speedup } (S) = \frac{1}{(1-P) + \frac{P}{N}}$$

$$\text{Max Speedup } (N \to \infty) = \frac{1}{1-P}$$

$$\text{Efficiency } (E) = \frac{S}{N}$$

Where:

- P = Parallel fraction of the program ($0 \leq P \leq 1$).
- $(1-P)$ = Serial fraction.
- N = Number of processors.

Code ([AmdahlLaw.java](file:///home/mint/Documents/COA/Week%206/AmdahlLaw.java))

```
public class AmdahlLaw {
    public static double speedup(double P, int N) {
        return 1.0 / ((1.0 - P) + (P / N));
    }

    public static void main(String[] args) {
        double[] parallelFractions = {0.5, 0.75, 0.9, 0.95, 0.99};
        int[] processors = {1, 4, 9, 16};
```

```
        for (double P : parallelFractions) {
            double maxSpeedup = 1.0 / (1.0 - P);
            System.out.printf("%nParallel fraction P = %.2f, Max Speedup =
%.4f%n", P, maxSpeedup);

            for (int N : processors) {
                double S = speedup(P, N);
                double E = S / N;
                System.out.printf("Processors: %2d | Speedup: %.4f |
Efficiency: %.4f%n", N, S, E);
            }
        }
    }
}
```

Experiment 7: Page Replacement Algorithms

Given reference string: `7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1` with `3` page frames.

1. FIFO (First-In, First-Out)

- Replaces the page that was brought in earliest (maintained via a circular pointer: `pointer = (pointer + 1) % FRAME_COUNT`).

2. LRU (Least Recently Used)

- Replaces the page that has not been referenced for the longest period of time (tracked by recording access cycle/timestamp `lastUsed[frame] = i`).

Code ([`PageReplacement.java`](file:///home/mint/Documents/COA/Week%207/PageReplacement.java))

```
import java.util.Arrays;

public class PageReplacement {
    static final int[] REF = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2,
0, 1, 7, 0, 1};
    static final int FRAMES = 3;

    public static void fifo() {
        int[] frames = {-1, -1, -1};
        int ptr = 0, hits = 0, faults = 0;

        for (int page : REF) {
            if (contains(frames, page)) {
                hits++;
```

```
            } else {
                frames[ptr] = page;
                ptr = (ptr + 1) % FRAMES; // Circular FIFO replacement
                faults++;
            }
        }
        System.out.printf("FIFO: Faults = %d, Hits = %d, Fault/Hit Ratio =
%.2f%n",
                faults, hits, (double) faults / hits);
    }

    public static void lru() {
        int[] frames = {-1, -1, -1};
        int[] lastUsed = {-1, -1, -1};
        int hits = 0, faults = 0;

        for (int i = 0; i < REF.length; i++) {
            int page = REF[i];
            int idx = indexOf(frames, page);
            if (idx != -1) {
                lastUsed[idx] = i; // Update timestamp
                hits++;
            } else {
                // Find frame with minimum (oldest) lastUsed index
                int victim = 0;
                for (int j = 1; j < FRAMES; j++) {
                    if (lastUsed[j] < lastUsed[victim]) victim = j;
                }
                frames[victim] = page;
                lastUsed[victim] = i;
                faults++;
            }
        }
        System.out.printf("LRU: Faults = %d, Hits = %d, Fault/Hit Ratio =
%.2f%n",
                faults, hits, (double) faults / hits);
    }

    static boolean contains(int[] arr, int val) { return indexOf(arr, val)
!= -1; }
    static int indexOf(int[] arr, int val) {
        for (int i = 0; i < arr.length; i++) if (arr[i] == val) return i;
        return -1;
    }

    public static void main(String[] args) {
        fifo();
        lru();
    }
}
```

Experiment 8: Cache Direct Mapping

Architecture & Bit Calculations

In a direct-mapped cache:

- $\text{No. of Main Memory Blocks} = \frac{\text{Main Memory Size}}{\text{Block Size}}$
- $\text{No. of Cache Lines} = \frac{\text{Cache Memory Size}}{\text{Block Size}}$
- $\text{Block Index Bits } (BI) = \log_2(\text{Blocks})$
- $\text{Cache Line Index Bits } (CI) = \log_2(\text{Lines})$
- $\text{Tag Bits } (TI) = BI - CI$

Mapping Formula

For every Main Memory Block Index (BI):

- $\text{Cache Index } (CI) = BI \pmod{\text{No. of Lines}}$
- $\text{Tag Index } (TI) = \lfloor BI / \text{No. of Lines} \rfloor$

Code ([directMapping.c](file:///home/mint/Documents/COA/Week%208/directMapping.c))

```c
#include <stdio.h>
#include <math.h>

void printBinary(int n, int bits) {
    for (int i = bits - 1; i >= 0; i--) {
        printf("%d", (n >> i) & 1);
    }
}

int main() {
    int mm_size, cm_size, block_size;
    printf("Enter Main Memory, Cache, and Block size (bytes): ");
    scanf("%d %d %d", &mm_size, &cm_size, &block_size);

    int noOfBlocks = mm_size / block_size;
    int noOfLines  = cm_size / block_size;

    int bi_bits = (int)log2(noOfBlocks);
    int ci_bits = (int)log2(noOfLines);
    int ti_bits = bi_bits - ci_bits;

    printf("\nBI Bits: %d | CI Bits: %d | TI Bits: %d\n", bi_bits, ci_bits,
ti_bits);
    printf("| Block Index (BI) | Cache Index (CI) | Tag Index (TI) |\n");

    for (int bi = 0; bi < noOfBlocks; bi++) {
        int ci = bi % noOfLines;
        int ti = bi / noOfLines;
```

```
            printf("| "); printBinary(bi, bi_bits);
            printf(" | "); printBinary(ci, ci_bits);
            printf(" | "); printBinary(ti, ti_bits);
            printf(" |\n");
        }
        return 0;
    }
```

Experiment 10: Pipelining Simulation

Theory & Formulas

A standard 5-stage RISC processor pipeline:

1. **IF**: Instruction Fetch
2. **ID**: Instruction Decode / Register Fetch
3. **EX**: Execute
4. **MEM**: Memory Access
5. **WB**: Write Back

Equations

- **Non-pipelined execution time**: $T_{np} = n \times k$ clock cycles (where n = instructions, k = stages = 5).
- **Ideal pipelined execution time**: $T_p = k + (n - 1)$ cycles.
- **Actual execution time with stalls**: $T_{\text{actual}} = T_p + \text{stalls}$.
- **Speedup**:

$$S_{\text{ideal}} = \frac{T_{np}}{T_p}, \quad S_{\text{actual}} = \frac{T_{np}}{T_{\text{actual}}}$$

- **Efficiency**:

$$\eta = \frac{S}{k} \times 100\%$$

Hazard in this simulation:

- I_1: `ADD R1, R2, R3` (writes result into R_1 in stage WB, cycle 5).
- I_2: `SUB R4, R1, R5` (needs value of R_1 in stage EX).
- Result: I_2 must **stall for 2 cycles** in EX until I_1 finishes WB.

Code ([pipelining.java](file:///home/mint/Documents/COA/Week%2010/pipelining.java))

```
public class pipelining {
    public static void main(String[] args) {
        int n = 3; // 3 instructions
```

```
        int k = 5; // 5 pipeline stages (IF, ID, EX, MEM, WB)
        int totalStalls = 2;

        int nonPipelinedCycles  = n * k;
        int idealPipelinedCycles = k + (n - 1);
        int actualPipelinedCycles = idealPipelinedCycles + totalStalls;

        float idealSpeedup   = (float) nonPipelinedCycles /
idealPipelinedCycles;
        float idealEfficiency = idealSpeedup / k;

        float actualSpeedup   = (float) nonPipelinedCycles /
actualPipelinedCycles;
        float actualEfficiency = actualSpeedup / k;

        System.out.println("Non-pipelined cycles: " + nonPipelinedCycles);
        System.out.println("Ideal pipelined cycles: " +
idealPipelinedCycles);
        System.out.printf("Ideal Speedup: %.2f | Efficiency: %.1f%%%n",
idealSpeedup, idealEfficiency * 100);

        System.out.println("Actual pipelined cycles (with stalls): " +
actualPipelinedCycles);
        System.out.printf("Actual Speedup: %.2f | Efficiency: %.1f%%%n",
actualSpeedup, actualEfficiency * 100);
    }
}
```

Quick Cheat-Sheet for Writing in the Exam:

Exp	Key Concept	Memory / Register / Logic Trick
1.1	Hello World	`INT 21H` with `AH = 09H`, string ending in `$`
1.2	16-bit Addition	`NUM1 DW 1234H`, `MOV AX, NUM1`, `ADD AX, NUM2`
2.1 & 2.2	Block Transfer	`MOV CX, 10`, `CLD`, `REP MOVSW` (`DS:SI` to `ES:DI`)
3.1	String Length	`LODSB`, `CMP AL, '$'`, `INC CX`
3.2	Compare Strings	`REPE CMPSB`, check `JNE`
3.3	Palindrome	`SI = start`, `DI = end`, loop `LEN/2` comparing `[SI]` and `[DI]`
3.4	Reverse String	`SI = end`, `DI = start_of_new_buf`, `LOOP`
4	Booth's Algorithm	`(Q0, Q_1)`: `10` $\implies A - M$, `01` $\implies A + M$, Arithmetic Right Shift
5.1	Restoring Div	Shift Left $[A, Q]$, $A - M$; if $A < 0 \implies Q_0 = 0$ & $A + M$

Exp	Key Concept	Memory / Register / Logic Trick
5.2	Non-Restoring Div	Check sign first: $+ \implies -M, - \implies +M$; final fix if $A < 0$
6	Amdahl's Law	$S = 1/((1-P) + P/N), E = S/N$
7	FIFO & LRU	FIFO = circular index; LRU = minimum last-access index
8	Direct Mapping	$BI = \log_2(\text{Blocks}), CI = BI \pmod{\text{Lines}}, TI = BI/\text{Lines}$
10	Pipelining	$T_{np} = n \times k, T_p = k + n - 1$, Hazard stall = 2 cycles